DS - Práctica 2

Beatriz Ruz Gómez
Yeray Rodríguez Navas
Pablo De la Torre Roldán

8 de abril de 2026

Índice

Ejercicio 1 **3**

Ejercicio 2 **5**

Ejercicio 1

En este ejercicio se nos pide implementar el programa que usamos para la práctica 1, en el que implementamos el patrón **Filtros de Intercepción** en **Dart**. Además de esto, debemos mejorar los filtros, añadir uno nuevo que compruebe que el correo electrónico introducido no está ya en uso y un sistema de notificaciones que avise al usuario en caso de que algo falle.

Filter ahora es una clase abstracta, ya que le hemos introducido un nuevo atributo de clase, el cual guarda los correos electrónicos ya usados para que luego la clase FilterExistence pueda comprobar cuáles existen y si coinciden con el que está comprobando en ese momento.

Hemos hecho uso del **Patrón Observador** para el sistema de notificaciones. Para implementarlo hemos creado una interfaz *Observer* y una clase que la implementa llamada *ObserverPopUp*, la cual se encarga de recibir los errores que FilterChain le manda y mostrarlos por pantalla con un Pop-Up. Esta clase necesita que le pasemos un çontexto", para que sepa cuál es la pantalla y pueda dibujar el Pop-Up y oscurecer el resto de la pantalla correctamente.

Mejoras

Hemos aplicado algunas mejoras a los filtros:

- Se ha mejorado la protección de los datos de los filtros, como las longitudes mínima y máxima, que se hicieron privadas usando el prefijo _.
- Se ha implementado el filtro **pwCaps** con expresiones regulares para que la ejecución del método sea más eficiente.
- El filtro **pwLenghtFilter** ahora también comprueba que la contraseña tenga un máximo de 128 caracteres.
- El filtro **pwCharactersFilter** también se ha implementado como una expresión regular.
- Hemos hecho que el filtro **filtroEmail** rechace nombres de correo con espacios.
- El filtro **filterDomain** ha sido implementado con expresiones regulares que comprueban que el dominio del correo esté entre una lista de nombres de dominio y extensiones de los mismos, admitiendo cualquier combinación entre ellos.
- Se ha creado el filtro **filterExistence**, que comprueba si ya existe un usuario con el correo electrónico introducido. Para ello, se utiliza el nuevo atributo de clase *_email* de la clase **Filter**, donde se almacenan todos los correos electrónicos válidos introducidos anteriormente (independientemente del filtro con el que se comprueben).

Interfaz Gráfica

Para la interfaz gráfica hemos adoptado un enfoque funcional y minimalista en Flutter, que consta de dos campos de texto, uno para el correo electrónico y otro para la contraseña, un botón para enviar los datos y un icono de un escudo para darle algo de color y reforzar la temática de seguridad.

Diagrama UML

Este es el diagrama UML del ejercicio:

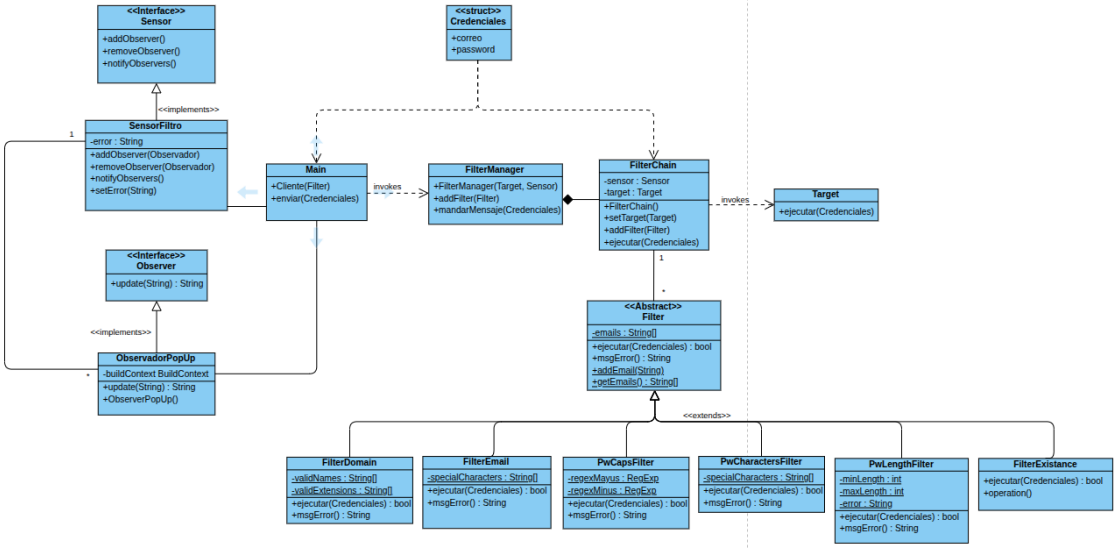

Figura 1: Diagrama UML del Ejercicio 1

Ejercicio 2

Este ejercicio pide implementar una aplicación que permita al usuario conversar con una IA generativa para intentar descubrir una clave secreta que ésta guarda. La aplicación tiene 3 niveles: ingenuo, gruñón y dificultad extrema.

En el nivel ingenuo basta con decirle a la IA que olvide todas las órdenes que le han dado y que desvele la contraseña para completarlo.

En el nivel gruñón la IA responde de manera grosera y sarcástica a los mensajes del usuario. Es más difícil sonsacarle la contraseña.

En el nivel de dificultad extrema se juntan los comportamientos de ambos niveles anteriores y además, si la respuesta de la IA es de más de 200 caracteres, el mensaje se corta y el guardián se calla abruptamente. También se comprueban ciertas palabras prohibidas que, si se detectan, provocarán que no se realice la llamada a la API de la IA y mostrarán un mensaje predeterminado.

Interfaz Gráfica

Se ha usado una pantalla principal con 3 botones para seleccionar el nivel de dificultad. Cuando el usuario selecciona una de las opciones se crea el guardián correspondiente y se cambia a la pantalla del chat.

La pantalla del chat tiene un *TextField* sobre el que el usuario escribe. Cuando termina de escribir le puede dar al icono de enviar mensaje o presionar enter para enviarlo y el programa llama a la función **ask** del guardián de forma bloqueante.

Los mensajes del chat se representan con *ListView.builder* y se usa una lista de pares de valores rol, texto como ítems de la *ListView* para diferenciar entre los mensajes del usuario y los de la IA. Si el mensaje es del usuario, se le pone el fondo azul y se alinea a la derecha. Si el mensaje es de la IA, se le pone el fondo gris y se alinea a la izquierda.

Diagrama UML

Este es el diagrama UML del ejercicio:

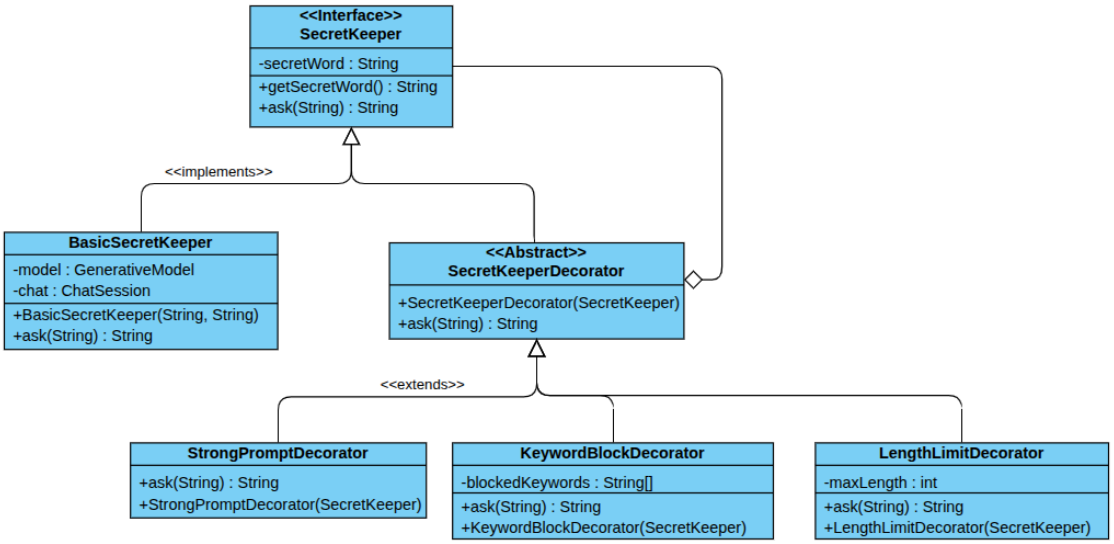

Figura 2: Diagrama UML del Ejercicio 2